Project report

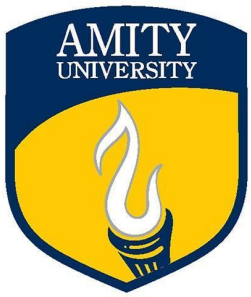
on

Machine-Generated Code Detection and Cyclomatic Complexity Analysis

Course code: CSE303

Submitted to

Amity University Uttar Pradesh, Noida



In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

In

Computer Science and Engineering

by

Name: Tanush Virmani

Enrollment number: A2305224371

Under the guidance of

Dr. Dipila Jain

Assistant Professor 2

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMITY SCHOOL OF ENGINEERING & TECHNOLOGY

AMITY UNIVERSITY UTTAR PRADESH, NOIDA

BATCH: 2024-2028

26 October 2026

1. INTRODUCTION

This project implements a local web application that estimates whether source code is human-written or machine-generated and independently measures cyclomatic complexity. The initial implementation used CodeBERT as a pretrained encoder, but lacked trained classification weights and a complete evaluation workflow. A runnable training and evaluation pipeline was added, followed by systematic experiments to address poor generalization.

The original 2,000-example CodeBERT experiment obtained 95.8% validation accuracy but only 43.1% accuracy on the 1,000-example public SemEval sample. This gap motivated language/class balancing, broader production-code coverage, function-fragment views and validation-based model selection. The selected deployed model combines lexical TF-IDF classification with structural boosting and randomized-tree classifiers. Its decision threshold is chosen using validation data.

The final reserved holdout contains 1,450 views from 454 previously uninspected parent files. The selected model obtained 79.66% view accuracy and 87.22% parent-file accuracy. The reused SemEval sample obtained 66.30%. These results describe different distributions and are reported separately.

Key Features of the Proposed System

A Gradio interface; validated local checkpoint loading; CPU-efficient and CodeBERT inference; data preparation, training, threshold selection and evaluation scripts; per-example predictions; confusion matrices; provenance metadata; and automated tests.

Project Workflow

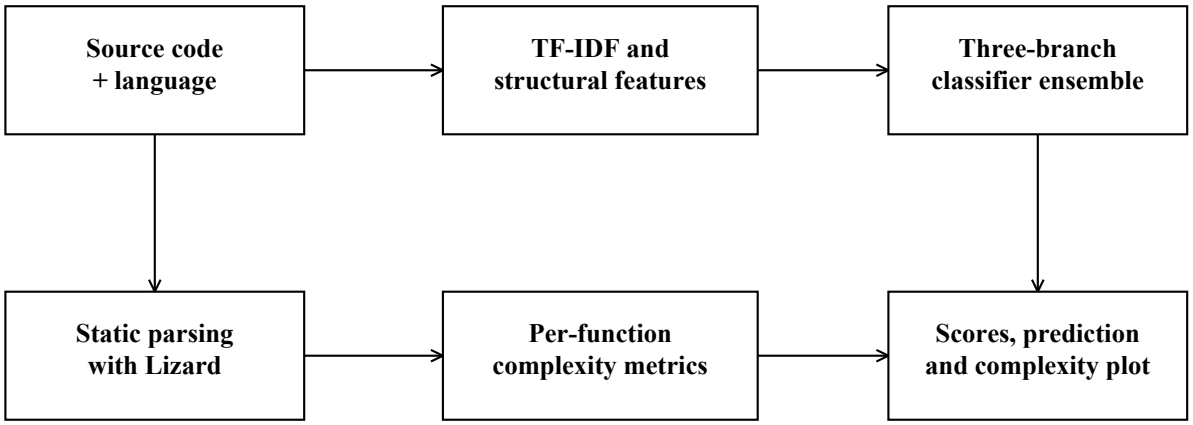


Figure 1. Code input, authorship classification and parallel complexity workflow.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Given a source-code snippet and its programming language, the detector assigns one of two labels: **0 - Human-written** or **1 - Machine-generated**. The label is a statistical prediction, not a determination of authorship. Code copied, edited, generated by templates, or jointly authored by people and AI may not fit a strict binary interpretation.

2.2 Objectives

1. Make the original project reproducible and prevent inference with randomly initialized classifier weights.
2. Calculate accuracy, precision, recall, F1 and confusion matrices from labeled, held-out examples.
3. Improve detection using broader training coverage and validation-selected models.
4. Expose predictions and cyclomatic-complexity analysis through a local application.
5. Preserve evidence, failed experiments and limitations rather than report only favorable numbers.

2.3 Scope

The academic demo supports the languages exposed by its current classifier and uses a binary authorship task inspired by SemEval Subtask A. It does not implement generator attribution, plagiarism detection or proof of misconduct. HTML can be classified in the broad demo, but cyclomatic-complexity analysis is unavailable for HTML.

Connection to Analysis and Design of Algorithms

The application compares two computational approaches: a transformer encoder and sparse lexical/structural classifiers. The project therefore considers preprocessing, feature extraction, model inference and memory/runtime trade-offs. The CPU ensemble provides a practical alternative when transformer fine-tuning is expensive on the available machine.

Cyclomatic complexity is a separate structural measure of control-flow paths. For a control-flow graph, the standard expression is $V(G) = E - N + 2P$, where E is the edge count, N the node count and P the number of connected components. The application uses Lizard to identify functions and count language-specific branch constructs. This measure is not the same as asymptotic runtime complexity.

3. PROPOSED SYSTEM DESIGN AND ARCHITECTURE

The system separates authorship inference from static complexity analysis. Both paths operate on the submitted code without executing it.

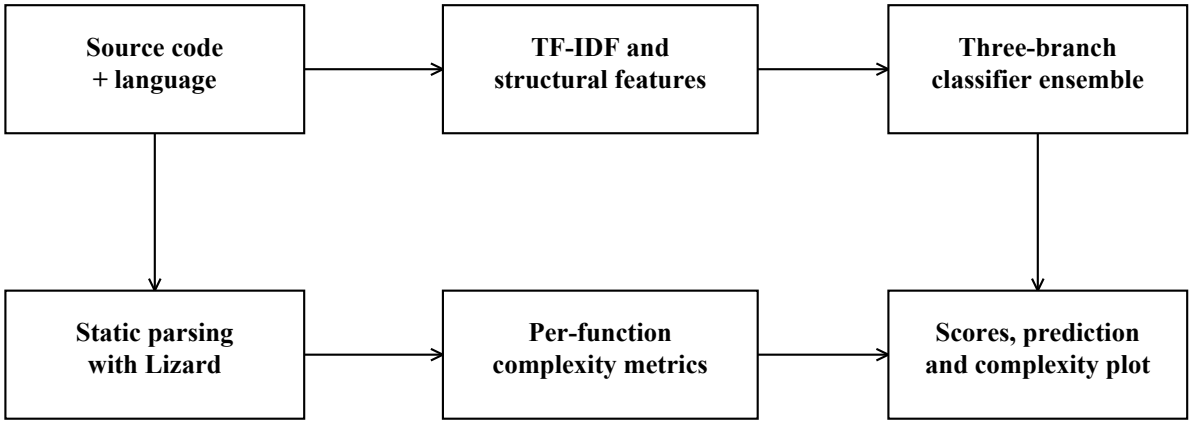


Figure 2. Proposed classifier and complexity-analysis architecture.

3.1 Major Modules

Module	Responsibility
app.py	Local Gradio UI, input handling, scores and complexity display
detector.py / model_utils.py	Shared inference, tokenization, checkpoint validation and decision scores
improve_model.py / fragment_views.py	CPU models, language/class balancing and fragment preparation
train_model.py	CodeBERT training, resumable checkpoints and optional layer freezing
evaluate_model.py	Metrics, leakage guards, predictions, parent aggregation and confusion matrix
model_selection.json	Recorded validation-only choice of the deployed model

Table 1. Major modules and their responsibilities.

3.2 System Data Flow

The same input is sent separately to Lizard for function-level complexity, lines of code and token counts. A horizontal chart shows complexity per function. Complexity analysis remains available when classifier weights are missing; the interface reports the detector as unavailable instead of inventing a prediction.

4. DATA SOURCES AND PREPROCESSING

4.1 Dataset Details

Dataset	Use and provenance
SemEval-2026 Task 13, configuration A	Official train/validation data and the 1,000-example public test sample. Training emphasizes Python, C++ and Java.
CodeMirage	Broader academic training coverage, including production-style code and normal/paraphrased AI examples. Original source: HanxiGuo/CodeMirage; license CC BY-NC-ND 4.0.
Final reserved files	454 remaining CodeMirage test parent files, not used in fitting or shared-validation selection.

Table 2. Data sources, roles and provenance.

4.2 Preparation and Duplicate Grouping

Empty or missing code/labels are rejected or removed at preparation time. Labels remain exactly 0 and 1. Exact code hashes prevent direct reuse across fitting and evaluation. A conservative normalized fingerprint also removes identifier names, literal values, comments and whitespace to group obvious renamed or reformatted code. Grouping does not establish that all semantically related programs or repository families are independent.

Function views are extracted using Lizard; a deterministic contiguous window is used when suitable functions are unavailable. Each view inherits its parent label. This broadens the input-length distribution, but inherited labels can be noisy when one file contains mixed or generated components.

4.3 Separation of Training, Validation and Evaluation

Training data fits model parameters. Validation data selects models and thresholds. A shared 1,000-example validation subset compares the completed final candidates. Reserved parent files are evaluated only after this choice is locked. Repeated code patterns matching fitting, tuning or previously inspected views are removed before the final holdout evaluation.

The SemEval public sample was inspected during the original failure analysis. Subsequent results on that sample are development rechecks, not fresh estimates. The broader track uses external training data and is not competition-compliant. Dataset revisions, file hashes and selection records are retained in manifests and training metadata.

5. MACHINE LEARNING MODEL AND METHODOLOGY

5.1 Selected Authorship Classifier

The selected model is **lexical_C1.0_ensemble**. Word/token TF-IDF uses 1-2 grams, with an 80,000-feature cap. Character TF-IDF uses 3-5 grams, with a 100,000-feature cap. Comment and literal normalization reduces some superficial differences. Structural statistics include line and identifier lengths, punctuation ratios, indentation and control-flow keyword frequencies.

The lexical branch uses logistic regression with C = 1.0. Two structural branches use histogram gradient boosting and an ExtraTrees ensemble. Their three machine-class scores are averaged. Candidate branches and ensembles are selected using validation macro F1. These feature caps are configuration limits, not claims about the final learned vocabulary size.

5.2 Recorded Model Settings

Setting	Value
Word/token TF-IDF	1-2 grams; maximum 80,000 features
Character TF-IDF	3-5 grams; maximum 100,000 features
Lexical classifier	Logistic regression, C = 1.0
Structural branches	Histogram gradient boosting and ExtraTrees
Ensemble rule	Mean of three machine-class scores
Decision threshold	0.55; selected using validation data
Model selection	Shared-validation macro F1

Table 3. Model configuration used by the academic demo.

5.3 Computational Considerations

Operation	Main cost / interpretation
TF-IDF transformation	Character/token scans for fixed n-gram lengths; sparse output
Logistic inference	Sparse dot product, proportional to active feature count
Tree inference	One traversal per tree; depends on tree count and depth
Transformer inference	Per-layer attention has a quadratic dependence on sequence length; feed-forward layers also contribute
Cyclomatic analysis	Static parsing and branch counting; no source code is executed

Table 4. Algorithmic cost and interpretation.

No controlled latency benchmark was conducted. Layer freezing reduces backpropagation work but does not remove the transformer forward pass. The local CPU was used for the recorded experiments.

6. EXPLAINABILITY AND EVIDENCE GENERATION

6.1 Decision Scores and Threshold

The deployed threshold is $t = 0.55$, chosen on validation data. For consistent UI decisions, raw machine score p is mapped to $p^* = p(1-t) / [p(1-t) + (1-p)t]$. A displayed machine score above 0.5 is equivalent to a raw score above t. These values are uncalibrated decision scores, not verified probabilities of authorship.

6.2 Cyclomatic Complexity Evidence

The same input is sent separately to Lizard for function-level complexity, lines of code and token counts. A horizontal chart shows complexity per function. Complexity analysis remains available when classifier weights are missing; the interface reports the detector as unavailable instead of inventing a prediction.

Per-function complexity, lines of code and token counts provide structural information alongside the authorship estimate. They do not explain the causal basis of the classifier decision or prove who wrote a program.

6.3 Responsible Interpretation

Predictions and uncalibrated confidence scores are estimates, not proof of authorship. Cyclomatic complexity measures control-flow structure, not asymptotic runtime. A human reviewer must interpret both outputs in context.

7. IMPLEMENTATION AND ENVIRONMENT

7.1 Software and Hardware

Component	Recorded implementation
Language	Python 3.11
Application	Gradio local web interface
Machine learning	scikit-learn; PyTorch and Hugging Face Transformers for CodeBERT experiments
Data and numerical work	pandas, NumPy, SciPy and joblib
Static analysis	Lizard
Visualization and report	Matplotlib and ReportLab
Hardware	Local CPU; no usable GPU in the execution environment

Table 5. Implementation environment.

7.2 Training and Inference Procedure

Prepare and validate labeled data; separate fitting and evaluation groups; generate permitted training views; fit candidate models; choose the model and decision threshold using validation data; save metadata and checkpoints; evaluate the locked model; expose inference and complexity analysis in Gradio.

7.3 Shared Inference and Checkpoint Handling

The shared detector interface supports both the CPU ensemble and trained CodeBERT checkpoints. Missing or incompatible classification weights are rejected. The original untrained-base-model fallback is removed. CodeBERT transfer uses first-and-last token handling; ensemble inference uses lexical and structural features from the full snippet.

8. RESULTS AND PERFORMANCE ANALYSIS

8.1 Exploratory Experiments

Experiments were preserved even when their public-sample results were worse. High scores on similar validation data did not guarantee performance across languages, domains or snippet types.

Exploratory experiment	Public SemEval recheck
Original CodeBERT; 2,000 official training examples	43.1%
Balanced 48,000-example CPU ensemble	29.7%
Broader full-file CPU ensemble	49.0%
Fragment-aware ensemble; validation-selected threshold	66.3%

Table 6. Recorded project information.

8.2 Completed Shared-Validation Comparison

Candidate	Accuracy	Macro F1
Fragment-aware CPU ensemble	79.5%	0.7948
CodeBERT transfer; 8 lower layers frozen	70.1%	0.7006
Imported external detector	55.3%	0.4723

Table 7. Recorded project information.

The fragment-aware ensemble was selected using the shared validation macro F1. CodeBERT transfer used 4,000 examples, 128 first-and-last tokens, one epoch, and a 5e-5 learning rate. The external detector was a comparison only; its precise training dataset is not documented by its authors, so its training/evaluation overlap could not be independently checked. It was not deployed.

The final reserved parent files were not used in this candidate selection. The model-selection record includes the validation-file SHA-256 hash and scores for all completed candidates.

8.3 Final Reserved-Holdout Results

Evaluation used **1,450 snippets/views** from **454 parent files**. The majority-class accuracy baseline is 52.41%. View accuracy is **79.66%**, macro precision 0.7982, macro recall 0.7983 and macro F1 **0.7966**.

Class	Precision	Recall	F1	Support
Human-written	0.7609	0.8348	0.7961	690
Machine-generated	0.8355	0.7618	0.7970	760

Table 8. Class-wise precision, recall, F1-score and support on the reserved holdout.

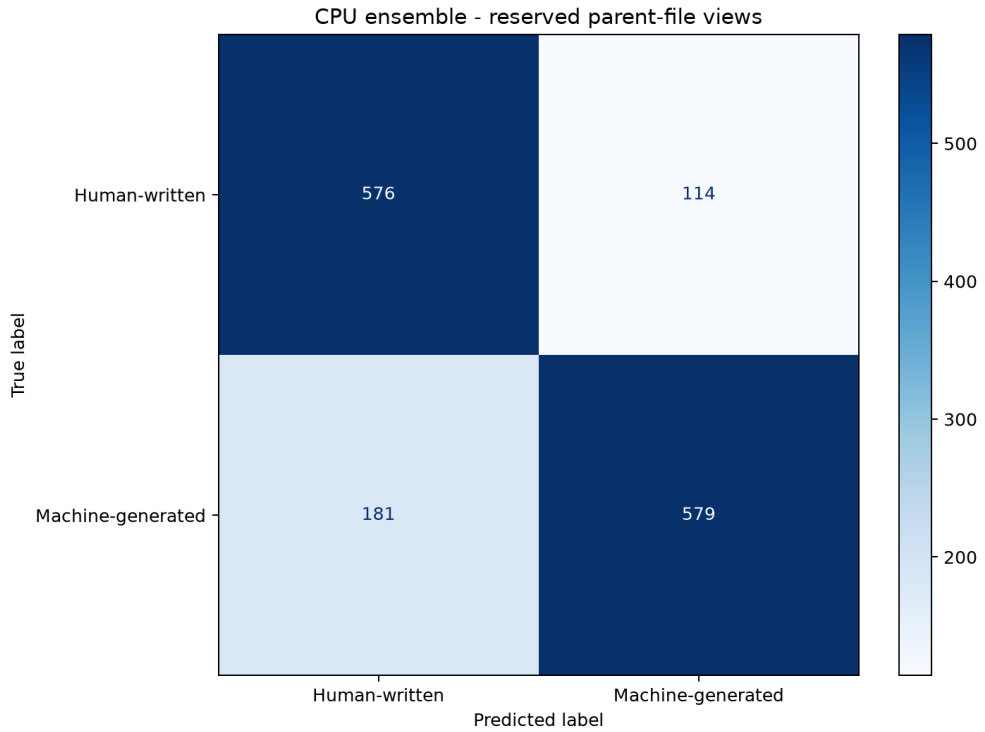


Figure 3. Confusion matrix on 1,450 reserved snippets/views from 454 parent files.

Averaging scores across each parent's views gives **87.22% parent-file accuracy** and 0.8721 macro F1. Views from one parent are correlated; these two levels must not be treated as independent sample counts.

This result measures the specified reserved CodeMirage-based task. It does not establish the same accuracy on all source code, generators, domains or future submissions.

8.4 Original SemEval Public Sample: Separate Result

The same 1,000-example public sample used in the original project now gives **66.30%** accuracy, macro precision 0.6367, macro recall 0.6952 and macro F1 0.6227. This is an exploratory recheck after development.

Class	Precision	Recall	F1	Support
Human-written	0.9000	0.6371	0.7460	777
Machine-generated	0.3733	0.7534	0.4993	223

Table 9. Class-wise results on the reused 1,000-example SemEval public sample.

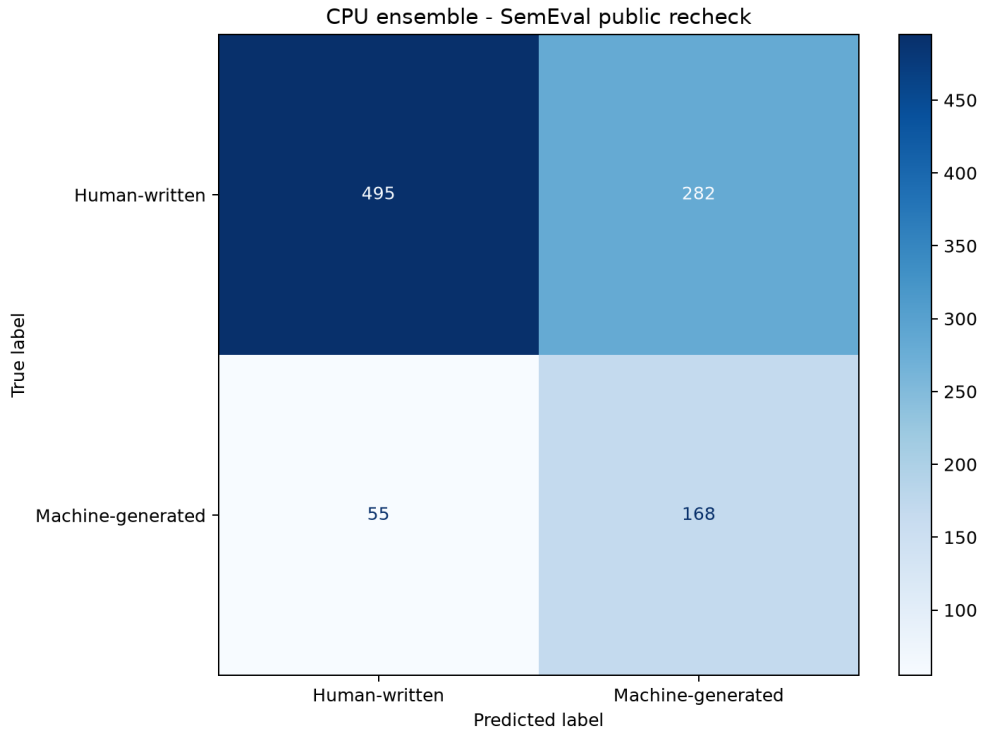


Figure 4. Confusion matrix for the SemEval public-sample development recheck.

Accuracy improved from 43.1% to 66.3%, but remains below both 75% and the sample's 77.7% majority-class accuracy baseline. The greater-than-75% objective is met on the separately described broader holdout (79.66%); it is **not achieved on this original SemEval sample**. The two benchmarks must not be merged or presented as directly interchangeable results.

9. ETHICAL CONSIDERATIONS

Authorship detection can produce false positives and false negatives. A human-written submission can be wrongly flagged, while AI-written code can be missed. The application is an academic demonstration and should not be used as a standalone basis for an accusation, grade penalty or consequential decision.

Source-level labels may be imperfect, and function views inherit their parent labels. Differences in language, domain, formatting, code length and generator family affect generalization. Confidence values are uncalibrated decision scores; high confidence is not equivalent to established authorship.

Downloaded datasets, local weights and raw prediction files remain separate from tracked code and summary metrics. The broader demo uses external CodeMirage training data, whose source and license are recorded. This configuration is not an official competition submission.

The report retains weak and unsuccessful experiments and distinguishes the reserved holdout from the previously inspected public sample. Thresholds must not be tuned on final test labels merely to exceed an accuracy target.

10. TESTING AND VALIDATION

10.1 Validation Plan

Test	Verified behavior
CSV labels and missing data	Invalid, fractional or missing labels are rejected; both classes are required
Checkpoint handling	Missing or incompatible trained classifiers are rejected
Leakage guard	Evaluation rejects snippets used in fitting or validation
Preprocessing and features	Normalization is consistent; structural features remain finite
Fragment generation	Extraction is deterministic and does not execute code
Decision scores	Threshold mapping preserves the intended class boundary
CodeBERT tokens	First-and-last token handling retains both ends within the token budget

Table 10. Functional validation checks implemented in the test suite.

10.2 Model Validation

Automated checks cover invalid/fractional/missing labels, both-class requirements, missing checkpoints, exact training/evaluation overlap, normalization, finite structural features, deterministic fragment extraction, threshold mapping, and first-and-last token handling. Saved predictions are checked against accuracy, macro F1 and confusion matrices. The local application was opened and tested through its browser interface.

10.3 Grouped Final Evaluation

The final reserved evaluation is performed after shared-validation selection is locked. Because several views may come from one parent file, the report includes both view-level metrics and a parent-level check based on averaged scores. Parent and view counts must not be treated as independent sample counts.

11. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

11.1 Advantages

The project provides a reproducible local workflow, trained-model guards, a CPU-efficient classifier, independent cyclomatic-complexity analysis, explicit data provenance, and recorded model-selection evidence. CodeBERT and unsuccessful variants remain available for comparison instead of being hidden.

11.2 Limitations and 11.3 Future Scope

Source-level labels can be noisy; function views may inherit imperfect labels. Normalized fingerprints cannot detect all semantic clones or shared repositories. Confidence is uncalibrated, and no detector output should be treated as proof of misconduct. Larger and better-grouped datasets, generator/repository-held-out validation, stronger code encoders, and carefully controlled GPU experiments are appropriate next steps. Thresholds must not be tuned on final test labels simply to exceed a target accuracy.

The current greater-than-75% result applies to the separately described reserved CodeMirage-based holdout. The original SemEval sample still scores 66.3%. This remaining distribution gap is a limitation, not an improvement that can be assumed from results on another dataset.

12. CONCLUSION AND REPRODUCIBILITY

12.1 Overall Results

The final deployed ensemble obtained 79.66% accuracy across 1,450 reserved views, with macro precision 0.7982, macro recall 0.7983 and macro F1 0.7966. Parent-file accuracy was 87.22% across 454 files. The reused SemEval public sample achieved 66.30%, compared with the original 43.1% baseline.

12.2 Conclusion

The project now supplies a working authorship-detection and complexity-analysis application with reproducible evaluation. Broader coverage and fragment-aware preparation improved the demonstration, while the independent public-sample gap shows why dataset-specific results must be interpreted carefully. The detector remains experimental and supports human review rather than replacing it.

12.3 Reproducibility Checklist

README.md and IMPROVEMENTS.md describe the environment and experiment commands. Run app.py from the project environment to use http://127.0.0.1:7860. active_model.json identifies the selected local checkpoint. The repository retains the original CodeBERT model and results as separate local artifacts. Raw datasets, caches and large local weights are ignored by Git.

Artifact	Purpose
README.md / IMPROVEMENTS.md	Environment, experiment commands, provenance and limitations
active_model.json	Selected local checkpoint
model_selection.json	Shared-validation selection and file hash
Training metadata and manifests	Data revisions, row counts, hashes and settings
Evaluation outputs	Metrics, per-example predictions and confusion matrices
Tests	Data, inference, grouping, threshold and tokenization checks

Table 11. Project artifacts supporting reproducibility.

13. REFERENCES

[1] SemEval-2026 Task 13 organizers. Task definition and restrictions. https://github.com/mbzuai-nlp/SemEval-2026-Task13

[2] DaniilOr/SemEval-2026-Task13. Official dataset. https://huggingface.co/datasets/DaniilOr/SemEval-2026-Task13

[3] HanxiGuo/CodeMirage. Dataset card and provenance. https://huggingface.co/datasets/HanxiGuo/CodeMirage

[4] Microsoft CodeBERT. Pretrained code encoder. https://github.com/microsoft/CodeBERT

[5] Lizard static complexity analyzer. https://github.com/terryyin/lizard

[6] scikit-learn. Feature extraction and classifier implementations. https://scikit-learn.org/stable/

[7] azherali/CodeGenDetect-CodeBert. External comparison model card. https://huggingface.co/azherali/CodeGenDetect-CodeBert